

Bases de la cybersécurité

Scripting Python — Module 1 · Durée : 1h

Cours Scripting Python

2026-09-13

Plan

- **1.** Qu'est-ce que la cybersécurité ? — **10 min**
- **2.** Le vocabulaire minimum — **10 min**
- **3.** Les 5 menaces à connaître — **15 min**
- **4.** Les 3 défenses à coder — **10 min**
- **5.** Cadre légal / éthique (FR) — **5 min**
- **6.** Pourquoi Python en cyber — **5 min**

1. La cybersécurité, c'est quoi ?

Protéger les systèmes, réseaux et données contre les attaques, les vols et les interruptions.

- Tout est connecté : hôpitaux, banques, usines, téléphones
- Une panne = vies, argent, confiance en jeu
- **Ransomware** : hôpital paralysé, rançon exigée
- **Phishing** : un simple clic vole un accès
- **Fuite** : millions de mots de passe revendus

1. Qui attaque, qui défend ?

Les attaquants

- Cybercriminels (ransomware, arnaques)
- États (espionnage, sabotage)
- Hacktivistes, insiders

Les défenseurs

- SOC / CERT : surveillent, répondent
- RSSI, pentesters, devs
- ...et bientôt vos scripts !

Domaines : réseau · web · système · crypto · gouvernance — on les croisera tous.

2. Vocabulaire (1/3) — CIA + AAA

CIA — ce qu'on protège

- **C**onfidentialité : l'accès est restreint
- **I**ntégrité : les données n'ont pas été modifiées
- **D**isponibilité : le service est opérationnel

AAA — comment on contrôle

- **A**uthentication : qui es-tu ? (login, MFA)
- **A**utorisation : droit de faire quoi ?
- **T**raçabilité (accounting) : qui a fait quoi ? (logs)

Exemple : login cassé = *Authentication* HS, log altéré = *Intégrité* + *Traçabilité* HS.

2. Vocabulaire (2/3) — les 5 mots à ne plus confondre

Terme	Définition courte	Exemple
Vulnérabilité	Faible exploitable	SQLi non filtrée, SSH mdp faible
Exploit	Code qui utilise la faille	Script brute-force SSH
Payload	Ce que l'exploit exécute	id; whoami, reverse-shell
IOC	Indice de compromission	IP, hash SHA256, domaine
TTP	Tactiques, Techniques, Procédures	Brute-force puis exfil DNS

2. Vocabulaire (3/3) — Kill Chain + MITRE ATT&CK

Kill Chain simplifiée (où se place votre script ?) : Reconnaissance → Accès initial → Exécution → Persistance → Exfiltration

- socket scan → **Reconnaissance**
- brute-force requests → **Accès initial**
- injection → **Exécution**
- hash / logs → **Détection**
- requêtes DNS anormales → **Exfiltration / C2**

MITRE ATT&CK = catalogue officiel des TTP. Retenez : **chaque script = une case ATT&CK.**

3. Les 5 menaces — vue d'ensemble

1. Brute-force / stuffing — SSH, web

2. Injection — SQLi, XSS, Commande

3. Malware / phishing — hash, URL

4. Scan / énumération — socket, ports

5. Exfiltration / C2 — DNS tunneling

Menace 1 — Brute-force / credential stuffing

- **Quoi** : deviner un mot de passe en testant systématiquement des milliers de combinaisons.
- **Brute-force** : on génère les essais (wordlist, dictionnaire). **Stuffing** : on rejoue des logins/mdp volés dans des fuites — les gens réutilisent leurs mots de passe !
- Cibles : SSH, RDP, formulaires web, API.
- Pourquoi ça marche encore : mots de passe courts, prévisibles (Azerty123), jamais changés.
- **Défenses** : MFA, limiter les tentatives (rate-limit), bannir après N échecs (fail2ban), mots de passe longs.
-  En TP (labo !) : automatiser les essais.  Côté Blue : repérer les vagues d'échecs dans les logs.

Menace 2 — Injection

- **Quoi** : une entrée utilisateur est interprétée comme du code → l'attaquant exécute ses propres ordres.
- **SQLi** : taper ' OR '1'='1 dans le login contourne l'authentification en truquant la requête SQL.
- **Commande** : un ; id glissé dans un champ s'exécute sur le serveur si l'entrée est concaténée dans un appel système.
- **XSS** : un script injecté dans une page vole le cookie de session des visiteurs.
- **Fix** : requêtes paramétrées, validation stricte des entrées, jamais de concaténation, messages d'erreur génériques (le détail aide l'attaquant).
-  En TP (labo !) : tester une SQLi.  Côté Blue : chercher les caractères suspects (' , ; , <script>) dans les logs.

Menace 3 — Malware / phishing

- **Quoi** : le phishing (mail piégé) est le 1er vecteur d'accès initial : un clic suffit.
- Indices : expéditeur imité, urgence artificielle (« compte bloqué »), pièce jointe ou lien inattendu, URL qui n'est pas le vrai domaine.
- **Malware** : ransomware (chiffre et rançonne), keylogger (vole la frappe), RAT (contrôle à distance).
- **Hash SHA256 = carte d'identité du fichier** : 1 bit change → hash totalement différent ; on compare aux bases d'IOC connus.
- **Défenses** : ne jamais ouvrir dans le doute, vérifier l'URL, antivirus, sauvegardes déconnectées.
-  Côté Blue : extraire hash + URL de chaque alerte, comparer aux IOC.

Menace 4 — Scan / énumération

- **Quoi** : cartographier une cible : quels ports sont ouverts, quels services et versions tournent.
- Ports ouverts = portes d'entrée potentielles ; la version du service dit s'il est vulnérable.
- Bannières et réponses verbeuses en révèlent trop : un serveur qui se présente aide l'attaquant.
- Étape **Reconnaissance** de la Kill Chain : tout commence par observer, bien avant d'attaquer.
- Scan furtif (lent, discret) vs agressif (rapide, bruyant et loggé).
-  En TP (labo uniquement !) : coder un scanner de ports.  Côté Blue : détecter les balayages dans les logs firewall.

Menace 5 — Exfiltration / C2

- **Quoi** : une fois dedans, l'attaquant vole des données (exfiltration) et garde le contrôle via un canal de commande (C2).
- **Beaconing** : le poste compromis « appelle » le serveur de l'attaquant à intervalles réguliers — discret car sortant.
- **DNS tunneling** : des données cachées dans des requêtes DNS, un protocole que personne ne bloque.
- **HTTPS vers domaine rare** : noyé dans le trafic web normal, presque invisible.
- Signaux Blue : volume anormal, domaines jamais vus, requêtes à heures fixes ou la nuit.
-  En TP : compter les requêtes par domaine et alerter sur l'inconnu.

4. Défenses (1/3) — principes

Défense en profondeur

Firewall + auth + logs + patch.

Moindre privilège

Droits minimum. Pas de root pour un scan !

Zero Trust

Chaque requête vérifiée + loggée.

4. Défenses (2/3) — Crypto utile + démo

- **Hash** (intégrité) : SHA256 = empreinte.
Vérifie mdp / logs. Irréversible.
- **Symétrique** : 1 clé (AES). Rapide, partage délicat.
- **Asymétrique** : publique/privée (RSA).
Échange + signatures.

```
import hashlib
# Démo 2 min : intégrité
def sha256_texte(t: str) -> str:
    return hashlib.sha256(t.encode()).hexdigest()

print(sha256_texte("motdepasse123"))
# vérif log : recalculer et comparer
```

Retenez : **hash** = détecter la modification, **chiffrement** = empêcher la lecture.

4. Défenses (3/3) — Hygiène

- Mots de passe longs + uniques + gestionnaire
- **MFA** partout (même en labo)
- Patching : OS + libs Python (pip)
- Sauvegardes testées
- Logs : qui, quoi, quand — et on les protège (hash !)

Le script le plus sûr est inutile sans hygiène de base.

5. Cadre légal / éthique — obligatoire (FR)

 **Pas de scan / test hors labo sans autorisation écrite.**

- **Art. 323-1 Code pénal** : accès frauduleux = délit (même « juste pour voir »).
- **Loi Informatique et Libertés + RGPD** : logs = données personnelles → minimiser, sécuriser, durées limitées.
- En classe : labo isolé, cibles dédiées, autorisation écrite, traçabilité.

Ça vous protège, vous et l'enseignant. Hacker éthique = hacker autorisé.

6. Pourquoi Python en cyber — le couteau suisse

● Red Team — attaquer (en labo)

- Scan ports / énumération (socket)
- Exploit / brute-force (requests, paramiko)
- OSINT (collecte, parsing, API)

● Blue Team — défendre / détecter

- Analyse de logs, détection d'IOC
- Réponse incident, durcissement
- Automatisation : hash, alertes, parsing

Python = rapide, lisible, bibliothèques nombreuses : **socket · requests · hashlib · re · paramiko**

Conclusion — et maintenant, on scripte !

- Python Red : socket → scan, requests → brute-force/injection (labo !)
- Python Blue : hashlib → intégrité, parsing logs → détection d'IOC
- Chaque TP = 1 étape Kill Chain / 1 case ATT&CK

Prochain module : prise en main Python → premier scanner + premier hash.

Règle d'or : **labo uniquement, autorisation écrite, on log tout.**